AXI4 MASTER WITH BURST TRANSEFER SUPPORT

Submitted in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

SUBMITTED BY

Name: Yelleti Harshavardhan

Reg. No.:2400040454



Department of Electronics and Communication Engineering

KONERU LAKSHMAIAH EDUCATION FOUNDATION

Green Fields, Vaddeswaram

Academic Year 2025–2026

CERTIFICATE

This is to certify that the project report entitled “AXI4 Master with Burst Transfer Support for DMA-to-Memory Data Movement” is a bona fide record of the project work carried out by Yelleti Harshavardhan (Roll No.: 2400040454), a student of the Department of Electronics and Communication Engineering, KL University, in partial fulfilment of the requirements for the award of the Bachelor of Technology (B.Tech.) degree in Electronics and Communication Engineering.

The project work was carried out under the guidance of Dr. M. Lakshmana Kumar and is submitted in accordance with the academic requirements of KL University.

Project Guide

Dr.M.LakshmanaKumar

Signature: _____

Head of the Department (HOD)

Name:_____

Signature: _____

Date:_____

Place: _____

DECLARATION

I hereby declare that the project report entitled **“AXI4 Master with Burst Transfer Support for DMA-to-Memory Data Movement”** has been prepared by me, **Yelleti Harshavardhan (Roll No.: 2400040454)**, under the guidance of **Dr. M. Lakshmana Kumar**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology (B.Tech.) degree in Electronics and Communication Engineering at KL University**.

I declare that the work presented in this report is based on the design, implementation, and functional verification carried out as part of this project. All technical information and external sources used in preparing this report have been or will be appropriately acknowledged and cited.

I further declare that this report has been prepared in accordance with the academic integrity policies and submission requirements of KL University.

Place: Vijayawada

Date: 11-oct-2026

Student Signature: Y.Harshavardhan

YelletiHarshavardhan

Roll No.: 2400040454

project Guide: Dr. M. Lakshmana Kumar

1.Abstract:

In this project, an AXI4 Master is designed and verified with burst transfers capability for memory transfers using Direct Memory Access (DMA). The primary goal of the design is to move data from source memory to destination memory efficiently, while at the same time minimizing the control overhead required to carry out each data transfer. The design comprises a 64-bit AXI4 data bus and a 32-bit AXI4 address bus. Burst transactions up to 16 beats are supported; each beat can transfer 8 bytes of data, therefore, making a maximum of 128 bytes.

The architectural design comprises a DMA controller, AXI4 burst calculation module, two separate AXI4 read and write modules, FIFO buffer for data buffering, outstanding transaction module, and AXI4 RAM based memory slave for verification. The burst calculation module takes into account the remaining data to be transferred, FIFO capacity, maximum burst length, and also the 4KB boundary of AXI4. This is to prevent the burst transaction from crossing a 4KB boundary.

The design was realized in Verilog using Xilinx Vivado 2025.2 for the XC7A100T FPGA device. The functional verification of the design was carried out using a Verilog testbench and AXI4 memory model. Successful verifications of both the normal 256 bytes DMA transfer and the boundary crossing 4KB DMA transfer were done and ALL TESTS PASSED reported in the simulation. The synthesis, placement and routing processes were also successful.

Keywords: AXI4, DMA, burst transaction, Verilog, FIFO, FPGA, Vivado, memory interface.

Table Of content:

S. No.	Contents	Page No.
-	Certificate	2
-	Declaration	3
-	Abstract	4
1	Introduction	6
2	Literature Review	7-9
3	Methodology	10
3.1	Overall Architecture and RTL Schematic	10-12
3.2	DMA Controller and Burst Calculation	12
3.3	AXI4 Read and Write Engines	12-13
3.4	FIFO and Transaction Tracking	13-14
3.5	AXI4 RAM Slave and FPGA Wrapper	14-15
3.6	Handshake Mechanism and Burst Equations	16-18
4	Functional Verification and Results	19
4.1	DMA Transfer Test Cases	19-20
4.2	Simulation Waveform	20
4.3	Resource Utilization and Timing Analysis	21
4.4	Placement and Routing Results	22
4.5	Power Analysis	23
5	Conclusion	24
6	Future Scope	25
7	References	26

1. INTRODUCTION

In modern FPGA-based systems, efficient data transfer between memory locations and processing units is essential for system performance. Processor-controlled transfers can increase workload and control overhead. Direct Memory Access (DMA) addresses this limitation by using a dedicated hardware controller to manage data movement with reduced processor intervention.

The Advanced eXtensible Interface 4 (AXI4), part of the AMBA protocol family, is widely used for high-performance communication in Field-Programmable Gate Array (FPGA) and System-on-Chip (SoC) designs. It supports independent read and writes channels, burst transactions, and VALID/READY handshaking, enabling efficient data movement between master and slave components [1, 2].

This project aims to design and implement an **AXI4 Master with Burst Transfer Support** for DMA-based memory-to-memory transfers. A key requirement is preventing burst transactions from crossing a 4 KB address boundary. The burst calculator determines the appropriate number of data beats by considering the remaining transfer size, boundary limit, available FIFO capacity, and configured maximum burst length.

The design uses a 32-bit address interface and a 64-bit data interface. Each beat transfers 8 bytes, with a configured maximum of 16 beats, equivalent to 128 bytes per burst. The architecture includes a DMA controller, burst calculator, independent AXI4 read and write engines, FIFO-based buffering, and outstanding transaction tracking.

The design was developed using Verilog and AMD Vivado Design Suite 2025.2 for the AMD Artix-7 XC7A100T FPGA. Functional verification was performed using a Verilog testbench and an internal AXI4 RAM-based memory slave. Both a 256-byte DMA transfer and a 4 KB boundary-handling test passed simulation. Synthesis, placement, and routing were also completed successfully.

The current implementation focuses on functional verification using internal RAM. Integration with an external DDR memory controller, such as an AXI-connected DDR/MIG interface, remains outside the scope of this project.

2.Literature Review:

2.1 Direct Memory Access (DMA)

Direct Memory Access (DMA) refers to a hardware-based technology for data transfer among memory addresses or between memory and peripheral interfaces with minimal processor intervention. In the case of FPGA-based systems, the DMA controller could manage data transfer independently of the processor when the configuration for data transfer is done. This approach would be useful especially in situations where many data transfers are needed. In this project proposal, the DMA controller would control the source address, destination address, size of data transfer, and transfer status.

2.2 AXI4 Protocol and Burst Transactions

The Advanced eXtensible Interface 4 (AXI4) is part of the AMBA family of on-chip communication protocols. It supports memory-mapped communication through five independent channels: read address, read data, write address, write data, and write response. AXI4 uses VALID and READY signals to control transfers between a master and a slave. Its burst mechanism allows multiple data beats to be transferred after a single address transaction, reducing the need to issue a separate address transaction for every beat [1, 2].

The address and data widths used for this project are 32-bits and 64-bits respectively in the AXI4 Master Interface. In each data transaction, 8-bytes of full width are transferred. The system is set up in such a way that it can accommodate up to 16 beats as maximum burst length, which amounts to 128 bytes per burst [1].

2.3 FIFO Buffering and Data Flow Control

FIFOs (First-In, First-Out) buffers are used extensively in digital circuits to temporarily store the information that is exchanged among blocks which operate at different speeds. In DMA systems, buffering is used between the read and write engines to take care of differences between the generation of data and its consumption. The handshake protocol between AXI4

VALID/READY signals allows the transfer to be suspended when the recipient is not ready to accept it [1, 2].

The new design features an axis_fifo buffer placed between the AXI4 read and write channels. The information read from the source memory is stored in the FIFO buffer and only then sent to the destination memory. The logic for calculating bursts takes into account the buffer size when calculating the number of bytes to request, thus decreasing the possibility of requesting too much information.

2.4 AXI4 Burst-Length and 4 KB Boundary Restrictions

However, AXI4 restricts burst transactions in order to ensure that proper addressing progression and interconnect work can take place. For example, an AXI4 burst transaction cannot span across a 4 KB address boundary. Furthermore, the number of beats in the transfer is encoded in the AxLEN as beats – 1. It is important to consider these restrictions in creating address and data transactions for read and write transactions [1].

The proposed axi4_burst_calc module will be able to calculate the allowable number of beats in a burst transaction through calculating the amount of transfer left, the number of beats that can still be performed before the next 4 KB boundary, the FIFO depth and the maximum number of beats allowed per burst. Thus, the burst size that will be chosen is limited by the most limiting condition. A transfer transaction that will exceed the 4 KB boundary will then be separated into different bursts.

2.5 Research Gap and Motivation for the Proposed Design

From the discussion about protocol requirements and design considerations, it follows that a DMA-based AXI4 Master should combine all the following functions: transfer control, burst generation, buffering and compliance with address boundaries. They should be coordinated in order to ensure reliable data transfer.

Such a requirement is fulfilled in the proposed project in the form of a modular structure including DMA controller, burst calculation block, separate AXI4 read and write engines, FIFO buffering and transaction management. The burst calculation block determines the

safe burst length taking into account such parameters as residual transfer length, boundary constraints, FIFO availability and maximum number of beats equal to 16. The internal AXI4 RAM based slave is used for transfer behavior verification during simulation.

Two main test cases are considered in the design analysis: 256-byte DMA transfer and a transfer starting from address 0x0FE0 where the handling of 4 KB boundary at 0x1000 address is required. Both tests were performed successfully in simulation. These results serve as project specific confirmation of the implemented functionality but do not prove any performance superiority to other DMA designs and do not confirm an external DDR/MIG interface functionality.

3. METHODOLOGY

The AXI4 Master with Burst Transfer Support is designed using a modular DMA-based architecture in Verilog. The design is developed using AMD Vivado Design Suite 2025.2, targeting the Artix-7 XC7A100T FPGA. Its primary objective is to transfer data from a source address to a destination address using AXI4 burst transactions.

The design uses a 32-bit address bus and a 64-bit data bus. Each AXI4 data beat transfers 8 bytes, and the maximum burst length is configured as 16 beats, allowing up to 128 bytes to be transferred in a single burst.

The architecture consists of a DMA controller, burst calculator, AXI4 read engine, AXI4 write engine, FIFO buffer, outstanding transaction tracker, and an internal AXI4 RAM-based slave. The burst calculator considers the remaining transfer size, available FIFO capacity, maximum burst length, and 4 KB address-boundary restriction to determine the appropriate burst size.

The data flow of the design is:

Source Memory → AXI4 Read Engine → FIFO → AXI4 Write Engine → Destination Memory

The design is functionally verified using a Verilog testbench and an internal RAM-based AXI4 slave. The current implementation does not include an external DDR/MIG interface.

3.1 Overall Architecture and RTL Schematic

The proposed AXI4 Master with Burst Transfer Support uses a modular DMA-based architecture to transfer data from a source memory address to a destination memory address. The DMA controller manages the complete transfer by receiving the source address, destination address, and transfer size from the user interface. It determines the required burst sizes and coordinates the read and write operations.

The AXI4 read engine retrieves data from the source memory and stores it in the FIFO buffer. The AXI4 write engine then reads the buffered data and transfers it to the destination memory. The DMA controller coordinates these operations until the complete data transfer is finished.

The major RTL modules used in the design are listed below:

1. axi4_dma_top.sv – Top-level DMA integration module.
2. dma_controller.sv – Controls the DMA transfer sequence.
3. axi4_burst_calc.sv – Calculates the appropriate AXI4 burst length.
4. axi4_read_engine.sv – Manages AXI4 read transactions.
5. axi4_write_engine.sv – Manages AXI4 write transactions.
6. axis_fifo.sv – Provides temporary data buffering between read and write operations.

7. outstanding_tracker.sv – Tracks outstanding transactions and their completion status.
8. axi4_ram_slave.sv – Provides an internal AXI4-compatible RAM interface for verification.
9. axi4_dma_fpga_top.sv – Integrates the DMA design with the FPGA-level interface.

This modular architecture separates control, burst calculation, data transfer, buffering, and transaction tracking into individual components. This organization makes the design easier to understand, verify, and maintain.

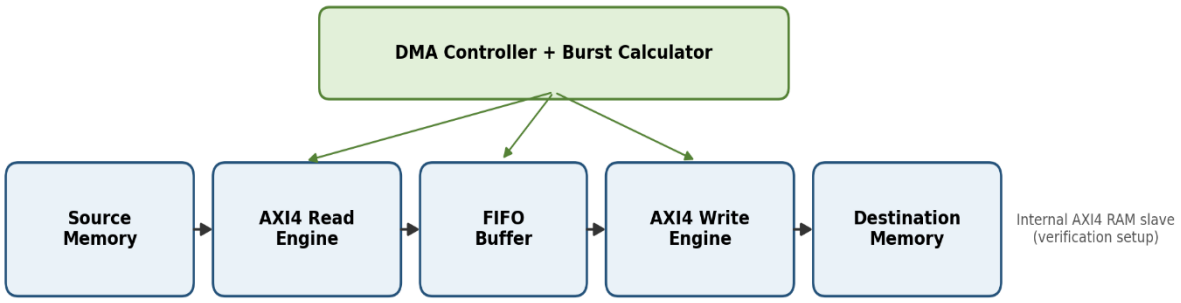
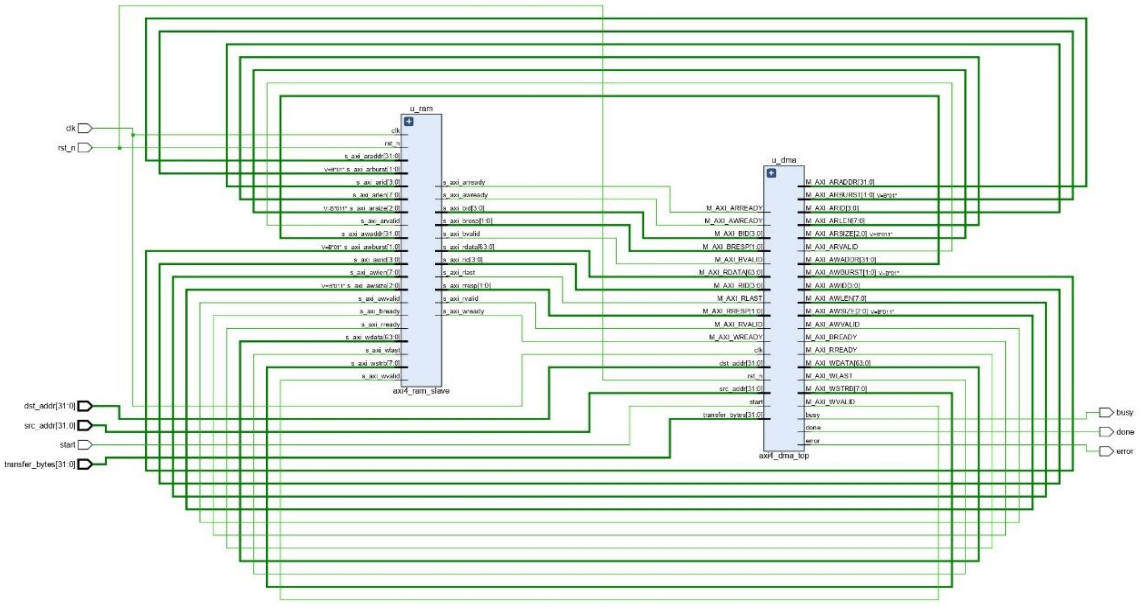


Figure 3.1: Conceptual Architecture of the AXI4 DMA System



3.2: RTL Architecture of the AXI4 Master with Burst Transfer Support

Table 3.1: Design Parameters and Configuration

Parameter	Configured value
RTL language	Verilog
Tool	AMD Vivado 2025.2
Target FPGA	Artix-7 xc7a100tcsg324-1
Address width	32 bits
Data width	64 bits
ID width	4 bits
FIFO depth parameter	512
Maximum burst length	16 beats
Bytes per beat	8 bytes

3.2 DMA Controller and Burst Calculation — Continued

4 KB Boundary Handling

An important feature of the design is handling the AXI4 4 KB address-boundary restriction. An AXI4 burst must not cross a 4 KB address boundary. Therefore, when the current address approaches the end of a 4 KB region, the burst calculator reduces the burst length to ensure that the transaction remains within the boundary.

For example, if a transfer begins near the end of a 4 KB memory region, the first burst is limited to the number of data beats that can fit before the boundary. The remaining data is transferred using subsequent bursts.

This approach prevents the DMA master from generating an invalid burst transaction. Boundary-aware burst calculation is particularly important for large transfers that span multiple 4 KB memory regions.

3.3 AXI4 Read and Write Engines

The AXI4 read and write engines manage data transfers between the DMA master and the memory interface. The read engine retrieves data from the source address, while the write engine transfers the received data to the destination address.

AXI4 Read Engine

The axi4_read_engine module manages the AXI4 read channels:

- AR channel: Read Address

- R channel: Read Data

The read engine receives the calculated source address and burst length from the control logic and generates the read address transaction. After the slave accepts the request, the returned data beats are received through the R channel.

The received data is stored in the FIFO for subsequent transmission by the write engine. The read engine also manages the AXI4 VALID/READY handshake signals to ensure that data transfers occur only when both the master and slave are ready.

AXI4 Write Engine

The axi4_write_engine module manages the AXI4 write channels:

- AW channel: Write Address
- W channel: Write Data
- B channel: Write Response

The write engine obtains data from the FIFO and transfers it to the destination address. The write address and burst length are generated according to the burst information supplied by the control logic.

For each burst, the write engine transmits the required number of data beats and asserts the WLAST signal on the final beat. After the slave processes the write transaction, the write response is received through the B channel.

Together, the read and write engines provide the complete AXI4 data-transfer path, from source-memory access to destination-memory writing.

3.4 FIFO and Transaction Tracking

FIFO-Based Data Buffering

The axis_fifo module provides temporary data storage between the AXI4 read and write engines. The data flow is:

Read Engine → FIFO → Write Engine

The FIFO is necessary because read and write operations may proceed at different rates. If the read engine receives data faster than the write engine can transmit it, the FIFO temporarily stores the additional data. Similarly, the write engine can consume previously received data while waiting for new data.

The available FIFO capacity is also considered during burst calculation to prevent the selected burst from exceeding the available buffering capacity. This supports backpressure handling and helps maintain a continuous data flow.

Outstanding Transaction Tracking

The outstanding tracker module tracks transactions that have been issued but have not yet completed. AXI4 supports multiple outstanding transactions, allowing a master to issue transactions without waiting for every previous transaction to finish, subject to the implemented design's capabilities.

The tracking logic maintains information about active transactions and their completion status. This helps coordinate read and write operations and prevents the DMA controller from declaring the transfer complete before all required transactions have finished.

3.5 AXI4 RAM Slave and FPGA Wrapper

FPGA Integration Wrapper

The axi4_dma_fpga_top module provides the FPGA-level integration wrapper for the DMA system. It connects the DMA architecture to the internal AXI4 RAM-based memory system and exposes the main control signals required to operate and monitor the transfer.

The main interface signals include:

- Clock: Provides the timing reference for synchronous operations.
- Reset: Initializes the design to its reset state.
- Source Address: Specifies the starting address of the data to be read.
- Destination Address: Specifies the address where the data will be written.
- Transfer Size: Defines the total amount of data to be transferred.
- Start: Initiates the DMA transfer.
- Busy: Indicates that the DMA operation is in progress.
- Done: Indicates completion of the DMA transfer.
- Error: Indicates a transfer error, if detected by the design.

The wrapper provides a synthesizable FPGA-level representation of the design. The current implementation uses an internal RAM-based AXI4 slave for functional verification and integration. An external DDR memory controller, such as an AXI-connected DDR/MIG interface, is not included.

DMA Transfer Sequence

The complete DMA operation follows the sequence described below:

Step 1: The user provides the source address, destination address, and transfer size and asserts the start command.

Step 2: The DMA controller initializes the transfer and records the remaining amount of data.

Step 3: The burst calculator determines a suitable burst size based on the remaining transfer size, available FIFO capacity, maximum burst length, and 4 KB boundary restriction.

Step 4: The AXI4 read engine generates the read address transaction.

Step 5: The source memory returns the requested data beats, which are received by the read engine and stored in the FIFO.

Step 6: The AXI4 write engine obtains the buffered data from the FIFO.

Step 7: The write engine generates the destination address transaction and transfers the data using AXI4 burst transactions.

Step 8: The write response is received, and the transaction status is updated.

Step 9: The DMA controller updates the source address, destination address, and remaining transfer size.

Step 10: The process continues with subsequent bursts until the required data transfer is completed.

Step 11: The DMA controller asserts the done signal to indicate completion.

This sequence coordinates memory reads, temporary data buffering, memory writes, and transfer-status updates to complete the DMA operation.

3.6 Handshake Mechanism and Burst Equations

AXI4 Handshake Mechanism

The design follows the AXI4 VALID/READY handshake mechanism. A transfer takes place when both the VALID and READY signals are asserted simultaneously during a clock cycle.

The handshake condition is expressed as:

$$\text{Transfer} = \text{VALID} \wedge \text{READY}$$

The handshake mechanism is used across the five AXI4 channels as follows:

Table 3.2: AXI4 Channel Handshake Signals

AXI4 channel	**Handshake signals**	**Purpose**
Read Address	ARVALID and ARREADY	Accepts a read address request
Read Data	RVALID and RREADY	Transfers read data
Write Address	AWVALID and AWREADY	Accepts a write address request
Write Data	WVALID and WREADY	Transfers write data
Write Response	BVALID and BREADY	Accepts the write response

Burst-Calculation Equations

The burst-calculation logic determines the appropriate number of AXI4 data beats for each transaction. The calculation considers the remaining transfer size, available FIFO capacity, maximum burst length, and 4 KB address-boundary restriction.

For the current design, the data width is 64 bits. Therefore, each data beat transfers 8 bytes.

The parameters used in the calculation are defined as follows:

- D: Number of bytes per data beat (D = 8).
- R: Remaining number of bytes to be transferred.

- C: Available FIFO capacity, expressed in beats.
- M: Maximum configured burst length (M = 16 beats).
- A: Current byte address.
- B: Selected burst length in beats.

1. Remaining Transfer Beats

The number of complete beats remaining is:

Remaining = R / D

2. Beats Permitted by the 4 KB Boundary

The number of bytes remaining before the next 4 KB boundary is:

Bytes_boundary = 4096 - (A mod 4096)

The number of complete beats that fit before the boundary is:

B_boundary = (4096 - (A mod 4096)) / D

3. Burst-Length Selection

The burst calculator selects the smallest applicable limit:

B = min (B_remaining, B_boundary, C, M)

This calculation reduces the burst length when the remaining transfer size, boundary restriction, FIFO capacity, or configured maximum burst length limits the transaction.

4. AXI4 Burst-Length Encoding

The AXI4 address-length fields specify the number of beats minus one:

AWLEN = B - 1

ARLEN = B - 1

Here, AWLEN is used for write-address transactions and ARLEN for read-address transactions.

5. Maximum Burst Size

For the implemented 64-bit data path:

D = 64 / 8 = 8 bytes

Maximum burst size = 16 × 8 = 128 bytes

Thus, the configured maximum burst transfers 128 bytes per transaction.

6. Address and Remaining-Data Updates

After a burst of B beats is processed, the address is updated as follows:

A_next = A + (B × D)

The remaining number of bytes is updated as follows:

R_next = R - (B × D)

The source and destination paths maintain separate addresses. After each burst, the controller updates the relevant addresses and remaining transfer count so that subsequent bursts continue from the correct locations.

The current implementation assumes that the source address, destination address, and transfer size are aligned to the 64-bit (8-byte) data width. Therefore, the calculation operates on complete 8-byte data beats. Unaligned transfers and partial-beat handling are outside the scope of the current implementation.

4. FUNCTIONAL VERIFICATION AND RESULTS

The proposed AXI4 Master with Burst Transfer Support was functionally verified through simulation and subsequently synthesized, placed, and routed using AMD Vivado Design Suite 2025.2 for the Artix-7 XC7A100T FPGA. The results evaluate DMA data-transfer correctness, 4 KB boundary handling, FPGA resource utilization, timing performance, and routing completion

4.1 DMA Transfer Test Cases

Two major test cases were used to verify the functionality of the DMA design.

Test Case 1: 256-Byte DMA Transfer

In the first test, a 256-byte data block was transferred from the source memory location to the destination memory location.

The testbench initialized the source memory with known data and started the DMA transfer. The DMA controller generated the required read and write burst transactions. After completion, the destination memory contents were compared with the original source data.

Result: PASS

The destination data matched the source data, confirming correct DMA data movement.

Test Case 2: 4 KB Boundary Transfer

The second test was designed to verify the 4 KB boundary-handling mechanism.

The transfer started at address 0x0FE0, near the 0x1000 boundary. The DMA had to divide the transfer into multiple bursts to prevent a burst from crossing the 4 KB boundary.

The transfer completed successfully, and the destination data was compared with the source data.

Result: PASS

This confirms that the burst-calculation logic correctly handled the tested transfer near a 4 KB boundary.

Overall Simulation Result

Both test cases completed successfully, and the simulation output reported:

ALL TESTS PASSED

These results verify the tested functionality of the DMA controller, burst calculation, AXI4 read and write engines, FIFO buffering, and boundary handling.

4.2 Simulation Waveform

The simulation waveform provides a visual representation of the DMA transfer operation and the interactions between the control signals and data-transfer logic.

The waveform can be used to observe the transfer initiation, read and write handshakes, transfer progress, and completion indication. It complements the testbench results by showing signal-level behavior during simulation.

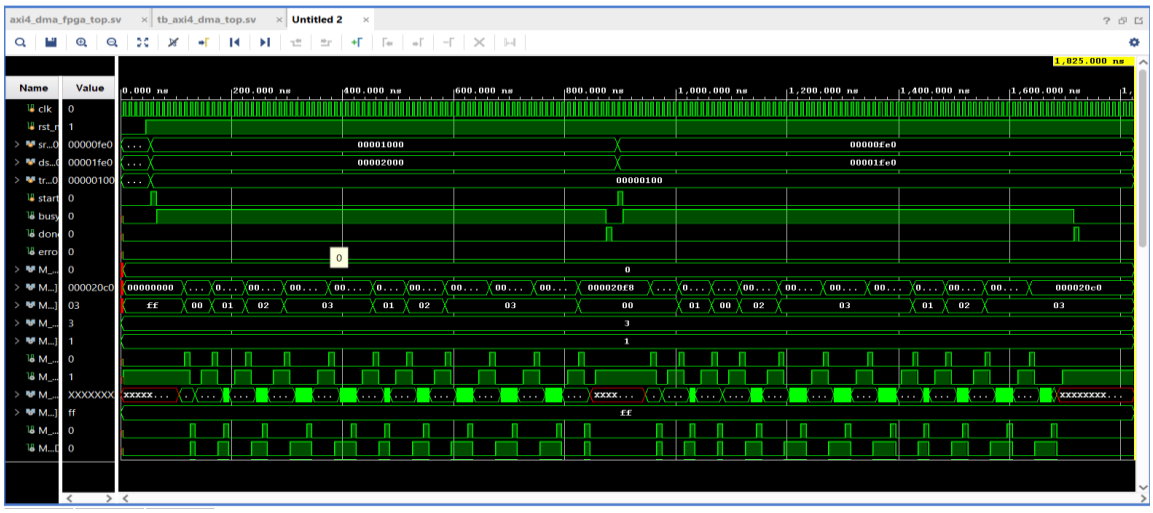


Figure 4.1: Simulation Waveform of the AXI4 DMA Transfer

4.3 Resource Utilization and Timing Analysis

FPGA Resource Utilization

After functional verification, the design was synthesized for the Artix-7 XC7A100T FPGA using AMD Vivado Design Suite 2025.2. The reported resource utilization is shown below.

Table 4.1: FPGA Resource Utilization

Resource	Used	Available	Utilization
Slice LUTs	285	63400	0.45%
Slice Registers	158	126800	0.12%
Block RAM	0	135	0.00%
DSPs	0	240	0.00%
Bonded IOB	62	210	29.52%
BUFGCTRL	1	32	3.13%

The design uses a small proportion of the available LUT and register resources. No dedicated Block RAM or DSP resources were reported as used by the synthesized design. The I/O utilization reflects the external ports defined by the FPGA integration wrapper.

Timing Analysis

The design was evaluated using a 100 MHz clock, corresponding to a clock period of 10 ns. The synthesis timing results are summarized below.

Table 4.2: Synthesis Timing Results

Parameter	Result
Clock Frequency	100 MHz
Clock Period	10 ns
WNS	+1.594 ns
TNS	0 ns
WHS	+0.142 ns
THS	0 ns
Failing Setup Endpoints	0
Failing Hold Endpoints	0

The positive Worst Negative Slack (WNS) and Worst Hold Slack (WHS), together with zero Total Negative Slack (TNS) and Total Hold Slack (THS), indicate that the reported synthesis timing checks met the specified constraints.

4.4 Placement and Routing Results

Following synthesis, the design was successfully placed and routed using Vivado.

The reported routing results are summarized below.

Table 4.3: Placement and Routing Results

Parameter	Result
Failed Nets	0
Unrouted Nets	0
Partially Routed Nets	0
Node Overlaps	0
Route Completion	Successful

The results indicate that placement and routing completed successfully, with no failed, unrouted, or partially routed nets reported. No node overlaps were reported in the implementation results.

The functional simulation, synthesis, and implementation results demonstrate that the design passed the two specified DMA test cases and completed the reported FPGA implementation flow. These results establish functional and implementation progress for the tested configuration; physical-board validation and external DDR/MIG integration are not claimed.

4.5 Power Analysis and Implemented Timing Summary

The Vivado power analysis estimates the total on-chip power as 0.219 W, including 0.111 W dynamic power and 0.108 W static power. The estimated junction temperature is 27.5 °C under the specified conditions. These values are software estimates and may vary during actual FPGA operation.

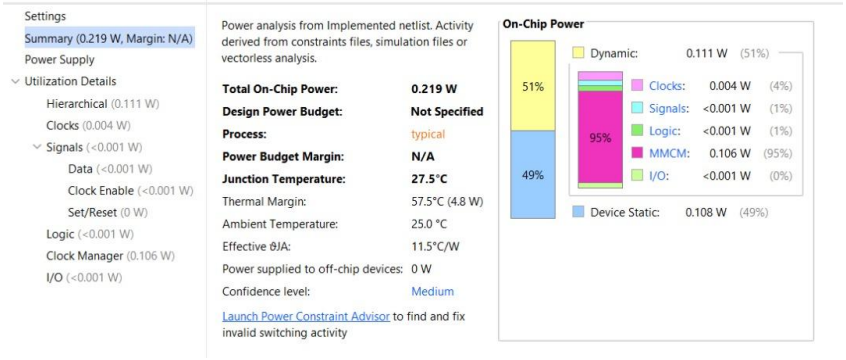


Figure 4.2: On-Chip Power Analysis

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.981 ns	Worst Hold Slack (WHS):	0.122 ns	Worst Pulse Width Slack (WPWS):	2.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	468	Total Number of Endpoints:	468	Total Number of Endpoints:	205

All user specified timing constraints are met.

Figure 4.3: Implemented Design Timing Summary

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LU
synth_1 (active)	**constrs_1**	**synth_design Complete!**												29
impl_1 (active)	**constrs_1**	**Running write_bitstream...**	**0.981**	**0.000**	**0.122**	**0.000**		**0.000**	**0.219**	**0**	**2 CW, 11 Warn**			28
impl_1_copy_1	constrs_1	Not started												
Out-of-Context Module Runs														
clk_wiz_0_synth_1	clk_wiz_0	Using cached IP results												

Figure 4.4: Vivado Implementation and Bitstream Status

The post-implementation timing summary reports WNS = +0.981 ns, TNS = 0 ns, WHS = +0.122 ns, and THS = 0 ns, with zero failing setup and hold endpoints. Vivado indicates that the user-specified timing constraints are met for the reported run. The implementation status screenshot shows write_bitstream running at the time of capture; it should not be described as a completed bitstream until the run finishes successfully.

5. CONCLUSION

The AXI4 Master with Burst Transfer Support was successfully designed and implemented as a DMA-based data transfer system using Verilog. The architecture integrates a DMA controller, burst calculation logic, AXI4 read and write engines, FIFO buffering, outstanding transaction tracking, and an internal AXI4 RAM-based memory interface.

The design uses a 32-bit address interface and a 64-bit data interface, supporting a configured maximum burst length of 16 beats, equivalent to 128 bytes per burst. The burst calculation logic determines the appropriate burst length based on the remaining transfer size, available FIFO capacity, maximum burst length, and the AXI4 4 KB address-boundary restriction.

Functional verification was performed using a Verilog testbench. Both the 256-byte DMA transfer and the test for 4 KB boundary handling completed successfully, confirming correct data movement and boundary-aware burst generation for the tested cases. The simulation concluded with the message “ALL TESTS PASSED.”

The design was successfully synthesized, placed, and routed using AMD Vivado Design Suite 2025.2 for the Artix-7 XC7A100T FPGA. Synthesis results showed low resource utilization, with 0.45% Slice LUT utilization and 0.12% Slice Register utilization. The reported synthesis timing checks also met the specified 100 MHz clock constraint, with positive setup and hold slack.

Overall, this project demonstrates a functionally verified AXI4 DMA Master that supports burst-based memory transfers while addressing the AXI4 4 KB boundary restriction. The implementation provides a foundation for further development, including external DDR/MIG integration, additional verification scenarios, and performance optimization. Physical FPGA-board validation and external DDR memory integration remain outside the scope of the current implementation.

6. FUTURE SCOPE

The current AXI4 Master with Burst Transfer Support provides a foundation for DMA-based memory-to-memory data transfers. The design can be further enhanced in the following areas:

1. External DDR Memory Integration: Integrate an AXI-connected DDR memory controller, such as the AMD MIG IP, to support data transfers between external memory locations.
2. Performance Optimization: Improve data throughput by optimizing burst scheduling, FIFO management, and read/write transaction coordination.
3. Support for Larger Transfers: Extend the design to handle larger DMA transfers efficiently by generating multiple consecutive bursts while respecting the AXI4 4 KB boundary restriction.
4. Advanced Transaction Management: Enhance outstanding transaction tracking to support improved transaction scheduling and increased transfer efficiency.
5. Expanded Verification: Add test cases for different transfer sizes, FIFO backpressure, reset behavior, error responses, and multiple boundary crossings to improve functional coverage.
6. Hardware Validation: Test the synthesized design on a physical Artix-7 FPGA board after assigning the required device pins, I/O standards, and clock constraints.
7. Power and Timing Optimization: Further optimize the RTL architecture to improve timing performance and evaluate power consumption for practical FPGA-based applications.

These improvements can extend the design toward a more flexible and efficient DMA subsystem suitable for future FPGA and System-on-Chip applications.

7. REFERENCES

[1] H. R. Archana, C. R. Byrareddy, and C. P. Narendra, "Design of an Efficient AXI-4 Protocol for High Speed SOC Applications on FPGA Platform," *Journal of Telecommunication, Electronic and Computer Engineering*, vol. 12, no. 3, pp. 61–68, 2020. Available: https://jtec.utem.edu.my/jtec/article/view/5774

[2] A. A. Khedkar and R. H. Khade, "High speed FPGA-based data acquisition system," *Microprocessors and Microsystems*, vol. 49, pp. 87–94, 2017. https://doi.org/10.1016/j.micpro.2016.11.006

[3] Z. Huang, S. Zhang, H. Gao, X. Zhang, and S. Yang, "A configurable multiplex data transfer model for asynchronous and heterogeneous FPGA accelerators on single DMA device," *Microprocessors and Microsystems*, 2020. https://doi.org/10.1016/j.micpro.2020.103174